

# Document 2 — Building & Accessing AI Agents

How to create AI agents in LangGraph, test them in Studio, and access the deployed agents from apps/code.

Server: langgraph.sudoxp.com · Repo: github.com/Harikarthik01/LangGraph-Development · Model: claude-sonnet-4-6 · Updated 2026-07-07

This is the **developer guide** for working on our LangGraph platform: understand the core concepts, **build a new agent in code**, test it visually in Studio, add multiple agents, and **access the running agents** via API from curl, Python, or JavaScript.

## §0 Contents

- §1 — Core concepts (graph, node, edge, state)
- §2 — Anatomy of our current agent
- §3 — Create a NEW agent (step by step)
- §4 — Hosting multiple agents
- §5 — Test an agent in Studio
- §6 — Access the deployed agents (curl / Python / JS)
- §7 — Threads & memory (multi-turn)

## §1 Core concepts

| Term         | What it is                                                                                         |
|--------------|----------------------------------------------------------------------------------------------------|
| Graph        | The agent itself — a flowchart of steps. This is what you build.                                   |
| Node         | A single step in the graph (e.g. "call the LLM", "look up data"). A Python function.               |
| Edge         | An arrow connecting nodes — defines the order/flow. Can be conditional (branch based on logic).    |
| State        | The data that flows through the graph (e.g. the list of messages). Each node reads and updates it. |
| START / END  | Special markers for where the graph begins and finishes.                                           |
| assistant_id | The name of a graph, used when calling the API to pick which agent to run.                         |
| Thread       | One ongoing conversation with saved history (stored in Postgres).                                  |

## §2 Anatomy of our current agent ( agent.py )

```
"""LangGraph agent backed by Claude (Anthropic)."""
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_anthropic import ChatAnthropic

class State(TypedDict):
    messages: Annotated[list, add_messages] # 1. the data that flows through

llm = ChatAnthropic(model="claude-sonnet-4-6") # 2. the model

def chatbot(state: State):
    return {"messages": [llm.invoke(state["messages"])]} # 3. a node = one step

builder = StateGraph(State) # 4. build the graph
builder.add_node("chatbot", chatbot)
builder.add_edge(START, "chatbot") # 5. wire the flow
builder.add_edge("chatbot", END)
graph = builder.compile() # 6. the server imports `graph`
```

**Read it as:** "Start → run the chatbot node (which calls Claude) → End." The langgraph.json file maps the name agent to this graph.

### §3 Create a NEW agent — step by step

Example: add a "researcher" agent with a custom system prompt.

#### Step 1 — Create `researcher.py`

```
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langchain_anthropic import ChatAnthropic
from langchain_core.messages import SystemMessage

class State(TypedDict):
    messages: Annotated[list, add_messages]

llm = ChatAnthropic(model="claude-sonnet-4-6")

def researcher(state: State):
    system = SystemMessage(content="You are a meticulous research assistant. "
                                   "Answer with sources and bullet points.")
    return {"messages": [llm.invoke([system] + state["messages"])]}

builder = StateGraph(State)
builder.add_node("researcher", researcher)
builder.add_edge(START, "researcher")
builder.add_edge("researcher", END)
graph = builder.compile()
```

#### Step 2 — Register it in `langgraph.json`

```
{
  "dependencies": [".",
  "graphs": {
    "agent": "./agent.py:graph",
    "researcher": "./researcher.py:graph"    <-- add this line
  },
  "env": ".env"
}
```

#### Step 3 — Test locally, then deploy

```
langgraph dev          # test in local Studio
# happy? →
git add . && git commit -m "add researcher agent" && git push
# on the server:
cd ~/LangGraph && git pull && sudo docker compose up -d --build
```

Now you have two agents on one server. Call them by name: `assistant_id: "agent"` or `assistant_id: "researcher"` .

### §4 Hosting multiple agents

One server hosts as many agents as you register in `langgraph.json` . Two patterns:

| Pattern                 | How                                                                                                                        |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Separate agents         | Each is its own graph file, registered in <code>langgraph.json</code> . Pick with <code>assistant_id</code> .              |
| Multi-agent (one graph) | Several sub-agents inside one graph (e.g. a supervisor delegating to workers). Built with extra nodes + conditional edges. |

### §5 Test an agent in Studio

| Environment            | How to open Studio                                                                              |
|------------------------|-------------------------------------------------------------------------------------------------|
| Local (building) 🌟     | Run <code>langgraph dev</code> → Studio opens at <code>...?baseUrl=http://127.0.0.1:2024</code> |
| Live server (deployed) | <code>smith.langchain.com/studio/?baseUrl=https://langgraph.sudoxp.com</code>                   |

In Studio: pick the agent from the dropdown → type a message → watch the graph run node-by-node → inspect the state → replay steps. Edits to code hot-reload locally.

## §6 Access the deployed agents

Base URL (secure): `https://langgraph.sudoxp.com` . Interactive docs: `/docs` .

 **Authentication required.** Running agents and accessing data needs the header `X-API-Key: <YOUR_API_KEY>` — requests without it get `401 Unauthorized` . Get the key from the team (password manager). **Public (no key):** `/ok` , `/info` , `/docs` (health/metadata only). **Protected:** `/runs` , `/threads` , and all agent operations.

**Using Studio against the live server:** in Studio → **Server connection settings**, set Base URL to `https://langgraph.sudoxp.com` and add a Custom Header `X-API-Key` = your key. (For everyday building, running Studio locally with `langgraph dev` is simpler.)

### A. Quick one-shot run (curl)

```
curl -s -X POST https://langgraph.sudoxp.com/runs/wait \
-H "X-API-Key: YOUR_API_KEY" \
-H "Content-Type: application/json" \
-d '{"assistant_id":"agent",
  "input":{"messages":[{"role":"user","content":"Hello!"}]}}'
```

### B. Python

```
import requests
r = requests.post("https://langgraph.sudoxp.com/runs/wait",
  headers={"X-API-Key": "YOUR_API_KEY"},
  json={
    "assistant_id": "agent",
    "input": {"messages": [{"role": "user", "content": "Hello!"}]},
  })
print(r.json()["messages"][-1]["content"]) # Claude's reply
```

### C. JavaScript (fetch)

```
const res = await fetch("https://langgraph.sudoxp.com/runs/wait", {
  method: "POST",
  headers: {
    "X-API-Key": "YOUR_API_KEY",
    "Content-Type": "application/json",
  },
  body: JSON.stringify({
    assistant_id: "agent",
    input: { messages: [{ role: "user", content: "Hello!" }] },
  }),
});
console.log((await res.json()).messages.at(-1).content);
```

| Endpoint                                  | What it does                                |
|-------------------------------------------|---------------------------------------------|
| <code>POST /runs/wait</code>              | Run and wait for the full reply (stateless) |
| <code>POST /runs/stream</code>            | Stream the reply token-by-token             |
| <code>POST /threads</code>                | Create a conversation thread (for memory)   |
| <code>POST /threads/{id}/runs/wait</code> | Run inside a thread (remembers history)     |
| <code>GET /assistants / docs</code>       | List agents / interactive API page          |

## §7 Threads & memory (multi-turn conversations)

For an agent that **remembers**, use a thread (state is saved in Postgres):

```
# reusable: export your key once
KEY="YOUR_API_KEY"

# 1. create a thread
TID=$(curl -s -X POST https://langgraph.sudoxp.com/threads -d '{}' \
-H "X-API-Key: $KEY" -H "Content-Type: application/json" \
| python3 -c "import sys,json;print(json.load(sys.stdin)['thread_id'])")
```

```
# 2. first message (saved)
curl -s -X POST https://langgraph.sudoxp.com/threads/$TID/runs/wait \
  -H "X-API-Key: $KEY" -H "Content-Type: application/json" \
  -d '{"assistant_id":"agent","input":{"messages":[{"role":"user","content":"My name is Hari."}]}}'

# 3. later message – the agent still remembers "Hari"
curl -s -X POST https://langgraph.sudoxp.com/threads/$TID/runs/wait \
  -H "X-API-Key: $KEY" -H "Content-Type: application/json" \
  -d '{"assistant_id":"agent","input":{"messages":[{"role":"user","content":"What is my name?"}]}}'
```

**Stateless vs stateful:** `/runs/wait` (no thread) forgets between calls — you send full history each time. `/threads/{id}/runs/wait` remembers automatically (persisted in Postgres). Use threads for real conversations.